

Operators

The *bitwise operators* `&`, `|`, `^`, and `~` operate on integers thought of as binary numbers or strings of bits. The `&` operator is bitwise AND, the `|` operator is bitwise OR, the `^` operator is bitwise exclusive-OR (XOR), and the `~` operator is a bitwise negation or complement. (`&`, `|`, and `^` are "binary" in that they take two operands; `~` is unary.) These operators let you work with the individual bits of a variable; one common use is to treat an integer as a set of single-bit *flags*. You might define the 3rd (2**2) bit as the "verbose" flag bit by defining

```
#define VERBOSE 4
```

Then you can "turn the verbose bit on" in an integer variable `flags` by executing

```
flags = flags | VERBOSE;
or
flags |= VERBOSE;
```

and turn it off with

```
flags = flags & ~VERBOSE;
or
flags &= ~VERBOSE;
```

and test whether it's set with

```
if(flags & VERBOSE)
```

The left-shift and right-shift operators `<<` and `>>` let you shift an integer left or right by some number of bit positions; for example, `value << 2` shifts `value` left by two bits.

The `?:` or *conditional* operator (also called the "ternary operator") essentially lets you embed an *if/then* statement in an expression. The assignment

```
a = expr ? b : c;
```

is roughly equivalent to

```
if(expr)
    a = b;
else
    a = c;
```

Since you can use `?:` anywhere in an expression, it can do things that *if/then* can't, or that would be cumbersome with *if/then*. For example, the function call

```
f(a, b, c ? d : e);
```

is roughly equivalent to

```
if(c)
    f(a, b, d);
else
    f(a, b, e);
```

(Exercise: what would the call

```
g(a, b, c ? d : e, h ? i : j, k);
```

be equivalent to?)

The comma operator lets you put two separate expressions where one is required; the expressions are executed one after the other. The most common use for comma operators is when you want multiple variables controlling a `for` loop, for example:

```
for(i = 0, j = 10; i < j; i++, j--)
```

A *cast operator* allows you to explicitly force conversion of a value from one type to another. A cast consists of a type name in parentheses. For example, you could convert an `int` to a `double` by typing

```
int i = 10;
double d;
d = (double)i;
```

(In this case, though, the cast is redundant, since this is a conversion that C would have performed for you automatically, i.e. if you'd just said `d = i`.) You use explicit casts in those circumstances where C does not do a needed conversion automatically. One example is division: if you're dividing two integers and you want a floating-point result, you must explicitly force at least one of the operands to floating-point, otherwise C will perform an integer division and will discard the remainder. The code

```
int i = 1, j = 2;
double d = i / j;
```

will set `d` to 0, but

```
d = (double)i / j;
```

will set `d` to 0.5. You can also ``cast to `void`'' to explicitly indicate that you're ignoring a function's return value, as in

```
(void)fclose(fp);
```

or

```
(void)printf("Hello, world!\n");
```

(Usually, it's a bad idea to ignore return values, but in some cases it's essentially inevitable, and the `(void)` cast keeps some compilers from issuing warnings every time you ignore a value.)

There's a precise, mildly elaborate set of rules which C uses for converting values automatically, in the absence of explicit casts.

The `.` and `->` operators let you access the members (components) of structures and unions.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)